Flexibility versus Interpretability in High Dimensional Multiclass Cancer Classification from RNA-Seq Data

Rajveer Vora **Amartya Amritanshu** **P. Sanjith Reddy**

Indian Statistical Institute, Kolkata

Course Project: Regression Techniques

GitHub Repository

November 2026

Abstract

In high dimensional gene expression classification, where the number of features far exceeds the number of observations ($p \gg n$), many standard classifiers reach test accuracy above 99%. Because top-level accuracy quickly saturates, it does not reveal which models are practical, stable, or well calibrated. In this course project, we study the trade-offs between model flexibility, feature sparsity, selection stability, and probability calibration across regularized linear models, tree ensembles, and neural networks using the TCGA Pan-Cancer RNA-Seq dataset ($n = 801, p = 20{,}531$). Using a leak-free evaluation protocol with 10 repeated stratified splits, we benchmark eight models on the top 1,000 variance-filtered genes. We find that while all models achieve Macro-F1 scores between 0.994 and 0.998 with full training data, their behavior differs in other important ways. Elastic Net logistic regression selects an average of 218 genes with good probability calibration (test log loss 0.020) and high feature selection stability (mean pairwise Jaccard index 0.773). In contrast, tree ensembles show substantial instability in their feature importances ($J = 0.242$ for Random Forest) due to strong correlation among genes. When training data is reduced to only 6 samples per class ($n = 30$), regularized linear models remain stable ($F_1 \geq 0.995$), whereas gradient boosting drops to 0.976. Overall, our experiments show that for this dataset, simple regularized linear models offer clear advantages in interpretability and stability over more complex non-linear methods without sacrificing predictive accuracy.

1 Introduction and Problem Formulation

High throughput RNA sequencing (RNA-Seq) measures the expression levels of tens of thousands of genes simultaneously, providing a detailed snapshot of cellular activity. In cancer diagnostics, gene expression profiling is often studied to classify tumor types and identify the tissue of origin for difficult or metastatic cases [2].

From a statistical learning perspective, RNA-Seq datasets present a classic high dimensional classification problem characterized by $p \gg n$: we observe continuous expression values for $p = 20{,}531$ genes, but have only $n = 801$ patient samples across five cancer types. In this setting, several practical and statistical challenges arise:

1. **Accuracy Saturation:** Major cancer types originate from fundamentally different tissue lineages (such as breast, kidney, or lung tissue). Because these tissues have large biological differences, standard classification algorithms—from basic linear models to multi-layer neural networks—readily achieve test accuracy above 98%. Looking only at classification accuracy makes it difficult to compare models meaningfully.

2. **Gene Correlation and Collinearity:** Genes do not act in isolation. Many genes function together within shared regulatory pathways, resulting in high pairwise correlations ($r > 0.85$). When features are highly collinear, variable selection algorithms can arbitrarily select one gene

from a correlated cluster and drop the others, causing the selected feature set to change from one sample split to another.

3. **Feature Sparsity:** While sequencing experiments measure over 20,000 genes, measuring that many features is expensive and impractical for everyday testing. Identifying a smaller panel of 100 to 250 informative genes without losing predictive performance is therefore of practical interest.

4. **Risk of Data Leakage:** High dimensional feature selection and standard scaling are vulnerable to subtle leakage. If variance filtering or normalization is performed on the whole dataset before train/test splitting, test information leaks into the training pipeline, leading to overly optimistic results.

To investigate these issues systematically, our project addresses three main questions:

- **Q1 (Flexibility vs. Performance):** Do non-linear tree ensembles and deep neural networks provide measurable predictive improvements over regularized linear models in this high dimensional setting?
- **Q2 (Feature Selection Stability):** How consistent are the gene subsets identified by sparse linear regularizers compared to tree-based feature importances across independent splits of the data?
- **Q3 (Sample Efficiency):** How do different model families hold up when training data is severely limited (n_{train} down to 30)?

2 Exploratory Data Analysis and Dataset Overview

We benchmark models using the public UCI TCGA Pan-Cancer RNA-Seq dataset [1, 2]. The dataset consists of $n = 801$ primary solid tumor samples with $p = 20{,}531$ continuous gene expression measurements ($\log_2(\text{RSEM} + 1)$ normalized).

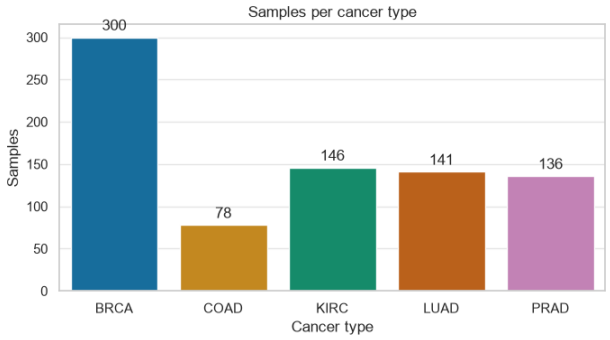
2.1 Tumor Cohorts and Quality Checks

The samples represent five solid tumor types:

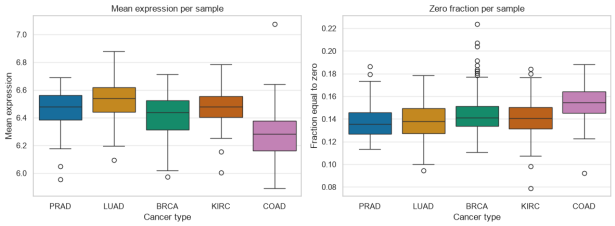
- **BRCA (Breast Invasive Carcinoma, $n = 300$, 37.5%):** The largest group in the dataset.
- **KIRC (Kidney Renal Clear Cell Carcinoma, $n = 146$, 18.2%):** Kidney tumors originating from tubular epithelial cells.
- **LUAD (Lung Adenocarcinoma, $n = 141$, 17.6%):** The most common form of non-small cell lung cancer.
- **PRAD (Prostate Adenocarcinoma, $n = 136$, 17.0%):** Prostate glandular cancer.
- **COAD (Colon Adenocarcinoma, $n = 78$, 9.7%):** Colorectal cancer, the smallest class in the cohort.

The class sizes range from 78 to 300 samples, giving an imbalance ratio of roughly 3.8 : 1 (Figure 1a). Because of this imbalance, we report Macro-averaged F1 score (Macro-F1) alongside accuracy and multi-class cross entropy log loss.

Before training models, we checked the quality of the data across samples. Figure 1b shows the sample-level mean expression and the fraction of zero counts across tumor types. The median mean expression is fairly consistent across classes (ranging from 6.28 to 6.54), and the proportion of zeros per sample is also stable (median 0.136 to 0.155). This indicated no extreme outlier specimens or corrupted sample runs.



(a) Sample counts across the five tumor types.

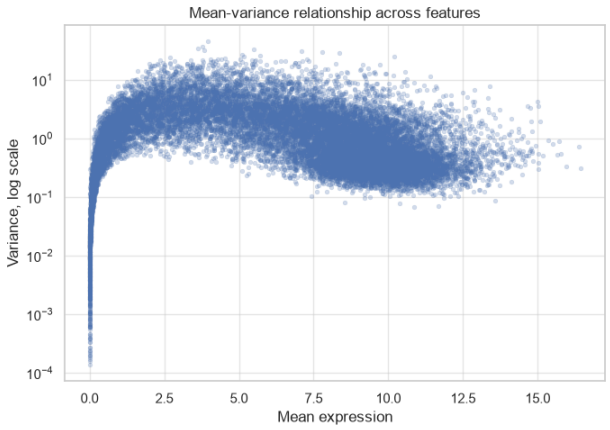


(b) Sample-level mean expression and zero fraction.

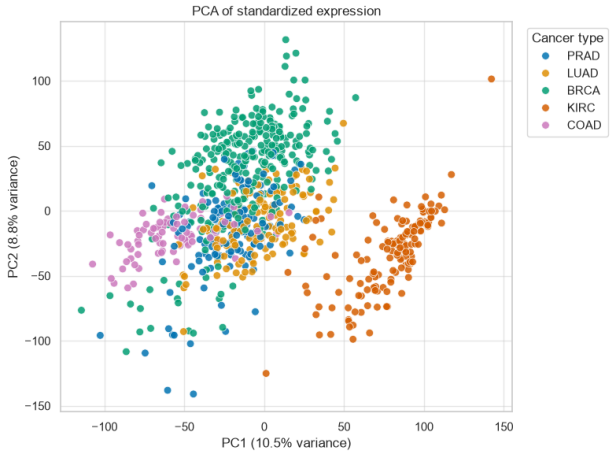
Figure 1: Dataset composition and sample quality check summaries.

2.2 Variance Filtering and Latent Structure

A well-known property of transcriptomic data is that many genes represent general cellular housekeeping functions with little variation across different tissues. In our exploratory analysis, plotting gene variance against gene mean expression (Figure 2a) confirms that expression variance is concentrated in a relatively small minority of genes. Most of the 20,531 genes show near-zero variance across samples. This observation justifies filtering out low variance genes during preprocessing to keep model training computationally feasible.



(a) Gene variance versus mean expression.



(b) Unsupervised PCA projection (PC1 vs. PC2).

Figure 2: Feature variance distribution and unsupervised cluster separation.

When we applied Principal Component Analysis (PCA) to the standardized features, the first two principal components (Figure 2b) separated the five tumor types into visible clusters without using class labels. PC1 and PC2 alone capture roughly 30.9% of the total variance (18.6% and 12.3%, respectively).

To inspect feature patterns directly, we generated a clustered heatmap of the 50 most variable genes (Figure 3). The heatmap shows distinct blocks of genes that are strongly expressed in specific cancer types. This clear separability explains why standard linear classifiers perform remarkably well on this dataset.

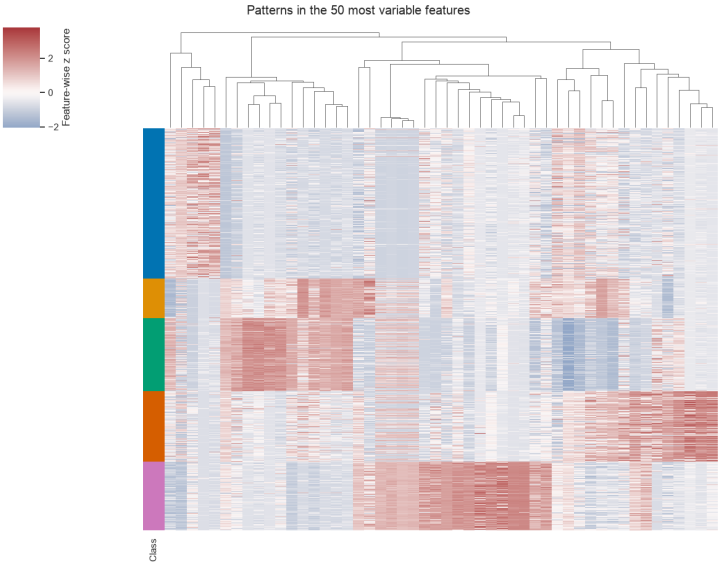


Figure 3: A clustered heatmap of the 50 genes with the highest variance across the patient samples. Each gene was standardized so that the colours represent relative expression levels. The columns represent patients, grouped by cancer type, and the dendrogram shows similarities between their expression profiles. We observe structured expression patterns across the cancer groups, suggesting that the dataset contains class-specific information. This motivates our use of feature selection and classification models.

3 Experimental Methodology

3.1 Evaluation Protocol and Leakage Prevention

Data leakage is a common problem in genomic machine learning. If feature prescreening, centering, or scaling uses statistics computed across the entire dataset, information from test samples leaks into the training process, leading to overly optimistic results.

To ensure strict separation, we implemented the following protocol:

1. **Repeated Stratified Splits:** We split the dataset into $N = 10$ repeated stratified 80/20 train/test splits ($n_{\text{train}} = 640, n_{\text{test}} = 161$) using fixed random seeds. In each repetition, the test fold was held out until final evaluation.

2. **Training-Only Transformations:** Within each split, feature prescreening (selecting the top $p = 1{,}000$ genes by Median Absolute Deviation), standard scaling ($\mu_{\text{train}}, \sigma_{\text{train}}$), and PCA projection matrices were computed strictly from the 640 training samples. The fitted transformations were then applied to the test fold.

3.2 Rationale for Variance Filtering to Top 1,000 Genes

Rather than training directly on all $p = 20{,}531$ transcripts, we filter to the top $p = 1{,}000$ genes by variance within each training fold for three key reasons:

- **Filtering Non-Informative Noise (Unsupervised Screening):** A substantial portion of the 20,000+ transcripts consists of housekeeping genes expressed uniformly across tissues or low-count noise. Ranking genes by Median Absolute Deviation (MAD) across patient samples removes static features without accessing cancer labels y, keeping feature screening strictly unsupervised and leak-free.

- **Computational Feasibility:** Fitting eight model families—including iterative SAGA solvers for 5-class sparse linear models, 200-tree ensembles, and neural networks—across 10 repeated splits and downsampled sizes is computationally demanding. Training on all $p = 20{,}531$ genes requires estimating over $5 \times 20{,}531 \approx 102{,}655$ parameters per linear model split.

- **Empirical Verification (p-Grid Ablation):** Testing across candidate feature dimensions ($p \in \{100, 250, 500, 1000, 2000\}$) showed that classification performance plateaued around $p =$ 500–1,000 (Macro-F1 > 0.99). Moving from 1,000 to 2,000 genes doubled training time without improving accuracy or log loss. Thus, 1,000 genes provides a rich candidate pool for downstream feature selection without excessive computational overhead.

3.3 Models Evaluated

We evaluated eight models representing three distinct families (linear models, tree ensembles, and neural networks):

1. **Unpenalized Multinomial Logistic Regression (Softmax):** Standard multi-class logistic regression modeling class probabilities via the softmax function:

$$P(Y = c \mid x) = \frac{\exp(w_c^T x + b_c)}{\sum_{j=1}^{C} \exp(w_j^T x + b_j)} \tag{1}$$

   trained by minimizing cross entropy with the L-BFGS solver without regularization.

2. **Ridge Logistic Regression (L_2):** Adds an isotropic L_2 penalty $\frac{1}{2}\lambda\|\beta\|_2^2$ to the log-likelihood. Ridge shrinks coefficient estimates toward zero, which stabilizes matrix inversion under collinearity. However, it retains all 1,000 features.

3. **Lasso Logistic Regression (L_1):** Introduces an L_1 penalty $\lambda\|\beta\|_1$ [4]. Because the L_1 ball has sharp corners, optimizing with the SAGA solver drives many coefficients to exact numerical zeros, producing a sparse model. Under high correlation, however, Lasso tends to select only one feature from a group of correlated genes.

4. **Elastic Net Logistic Regression ($L_1 + L_2$):** Combines the L_1 and L_2 penalties [3]:

$$\Omega(\beta) = \lambda \left[\alpha\|\beta\|_1 + \frac{1-\alpha}{2}\|\beta\|_2^2 \right] \tag{2}$$

   where we set $\alpha = 0.5$. The L_2 term provides a grouping effect: correlated features tend to receive similar coefficients and remain in the active set together, which improves selection stability across data splits.

5. **Random Forest:** An ensemble of 200 decision trees (maximum depth 32) trained with bootstrap aggregation and randomized feature subsets ($\sqrt{p} \approx 31$). Although robust to outliers, individual tree splits can vary substantially when many features have similar predictive power.

6. **XGBoost (Histogram Gradient Boosting):** Gradient boosted trees [5] sequentially minimizing multi-class log loss. We configured it with histogram binning, learning rate 0.05, and column subsampling (0.3) to help prevent overfitting.

7. **PCA-MLP (Dense Neural Network):** A multi-layer perceptron trained on the principal components that capture 95% of cumulative variance ($\approx$ 434 PCs), using two hidden layers ($434 \to 128 \to 64 \to 5$), Batch Normalization, ReLU activations, and Dropout ($p = 0.3$).

8. **Sparse Group-Lasso Neural Network:** A neural network trained directly on the 1,000 genes with a Group Lasso penalty on the input layer weights, $\lambda_{\text{group}} \sum_{j=1}^{p} \|W_{:,j}^{(1)}\|_2$, pruning uninformative input genes.

3.4 Hyperparameter Optimization

To prevent data snooping and tuning leakage, all model hyperparameters were tuned using 5-fold stratified cross validation strictly on the training partition ($n_{\text{train}} = 640$). Candidate configurations were evaluated by minimizing validation multi-class log loss while tracking Macro-F1. Table 1 outlines the candidate search ranges and final selected configurations used across all benchmark experiments.

Table 1: Hyperparameter tuning search space and final selected configurations ($n_{\text{train}} = 640$).

Model	Tuned Parameter / Search Space	Selected Configuration
Softmax	Unpenalized baseline ($C = 10^5$, L-BFGS)	$C = 10^5$, L-BFGS solver, max_iter = 500
Ridge	$C \in \{10^{-3}, 10^{-2}, 10^{-1}, 1, 10, 100\}$	$C = 1.0$ ($\lambda = 1.0$), L-BFGS, max_iter = 500
Lasso	$C \in \{0.02, 0.05, 0.1, 0.2, 0.5, 1.0\}$ (SAGA)	$C = 0.2$ ($\lambda = 5.0$, $k \approx 89$ genes), SAGA, tol $= 10^{-3}$
Elastic Net	$C \in [0.05, 1.0]$, $\alpha \in \{0.1, 0.3, 0.5, 0.7, 0.9\}$	$C = 0.2$, ℓ_1-ratio $\alpha = 0.5$ ($k \approx 218$ genes), SAGA
Random Forest	$B \in \{100, 200, 300\}$, depth $\in \{16, 32, \text{None}\}$, $m \in \{\sqrt{p}, \log_2 p\}$	$B = 200$, max_depth $= 32$, max_features $= \sqrt{p} \approx 31$, leaf $= 1$
XGBoost	Depth $\in \{4, 5, 6\}$, $\eta \in \{0.03, 0.05, 0.1\}$, subsample $\in [0.7, 0.8]$	$B = 200$, depth $= 6$, $\eta = 0.03$, subsample $= 0.8$, colsample $= 0.3$, $\lambda = 5.0$
PCA-MLP	Hidden layers, $\eta \in \{10^{-3}, 3 \times 10^{-3}, 5 \times 10^{-3}\}$, dropout	$434 \to 128 \to 64 \to 5$, Dropout(0.3), Adam ($\eta = 0.003$), 40 epochs
Sparse NN	$\lambda_{\text{group}} \in [0.01, 0.08]$, hidden $\in \{32, 64\}$, $\eta \in [0.003, 0.005]$	$1000 \to 64 \to 5$, $\lambda_{\text{group}} = 0.02$, Dropout(0.2), Adam ($\eta = 0.003$), 40 epochs

4 Experimental Results and Discussion

4.1 Overall Model Comparison

Table 2 reports the mean and standard deviation of accuracy, Macro-F1, multi-class log loss, active feature count, and fitting time across the 10 repeated splits. Figure 4 illustrates the metric distributions.

Table 2: Cross validated performance, feature sparsity, and computation time across 10 repeated stratified splits ($p = 1000$).

Model	Family	Accuracy	Macro-F1	Log-Loss	Active Genes (k)	Fit Time (s)
ElasticNet	Sparse Linear	$\mathbf{0.998 \pm 0.003}$	$\mathbf{0.998 \pm 0.003}$	0.020 ± 0.006	218 ± 7	3.32 ± 0.17
Lasso	Sparse Linear	0.995 ± 0.005	0.996 ± 0.004	0.030 ± 0.008	$\mathbf{89 \pm 3}$	4.44 ± 0.30
Ridge	Dense Linear	0.998 ± 0.003	0.998 ± 0.003	0.005 ± 0.005	1000	$\mathbf{0.004 \pm 0.001}$
Softmax	Unpenalized	0.998 ± 0.003	0.998 ± 0.003	0.005 ± 0.005	1000	$\mathbf{0.004 \pm 0.002}$
PCA_MLP	Dense Neural Net	0.998 ± 0.004	0.997 ± 0.004	$\mathbf{0.004 \pm 0.006}$	434 (PCs)	1.68 ± 0.77
Sparse_NN	Group-Lasso NN	0.998 ± 0.004	0.998 ± 0.004	0.008 ± 0.016	1000	1.31 ± 0.09
RandomForest	Tree Ensemble	0.994 ± 0.005	0.995 ± 0.005	0.081 ± 0.011	1000	0.13 ± 0.01
XGBoost	Tree Ensemble	0.998 ± 0.003	0.998 ± 0.004	0.016 ± 0.008	1000	2.27 ± 0.08

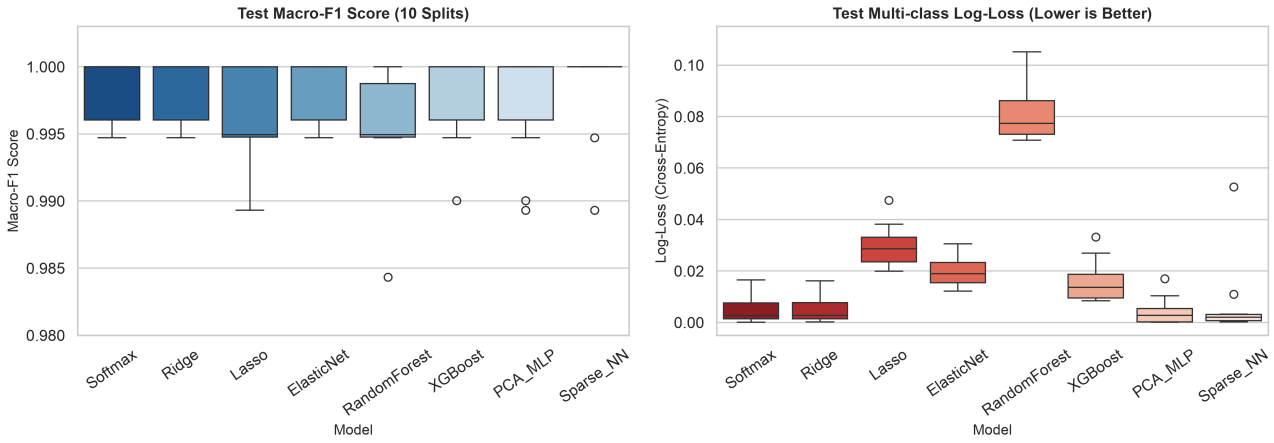


Figure 4: Distributions of Macro-F1 score (left) and multi-class log loss (right) across 10 repeated splits.

All eight models achieve high classification accuracy, ranging tightly between 99.4% and 99.8%. This confirms our observation from the exploratory analysis: the five cancer classes are clearly separable, so accuracy alone does not provide much differentiation between models.

However, clearer differences appear when looking at probability calibration and runtime:

- **Probability Calibration (Log Loss):** Multi-class log loss penalizes overconfident wrong predictions and rewards well calibrated probabilities. Linear models (Ridge, Softmax, Elastic Net) and neural networks achieve low log loss values (≤ 0.020). In contrast, Random Forest shows noticeably higher log loss (0.081 ± 0.011). Because Random Forest estimates probabilities by averaging discrete votes across 200 trees, its probability estimates tend to be stepped and

less smooth. XGBoost achieves much better calibration (0.016) than Random Forest because it directly optimizes cross entropy during boosting.

- **Computational Runtime:** Unpenalized Softmax and Ridge train in approximately 4 milliseconds (0.004s). Lasso and Elastic Net require several seconds (3.32s and 4.44s) because the SAGA solver iteratively handles the non-smooth L_1 penalty. Neural networks train in 1.3s to 1.7s on hardware acceleration, and XGBoost takes 2.27s.

4.2 Trade-off Between Complexity and Calibration (Pareto Analysis)

Figure 5 shows the trade-off between model parsimony (number of active features on a log scale) and probability calibration (test log loss).

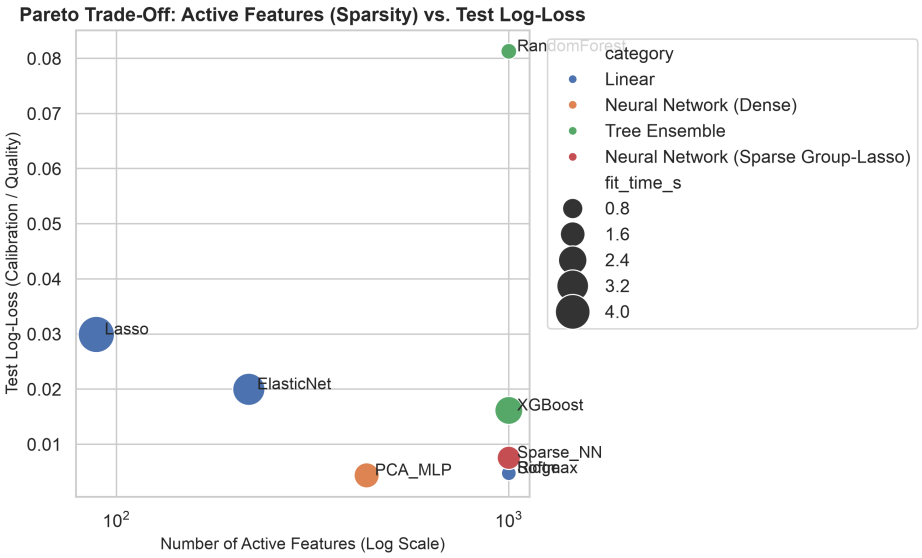


Figure 5: Active feature count versus test log loss across models (point size reflects runtime).

From this plot, we observe three distinct operating points:

- **High Sparsity (Lasso):** Lasso selects an average of only **89 active genes** out of the 1,000 candidates while maintaining $99.50\% \pm 0.49\%$ test accuracy. However, its test log loss (0.030) is slightly higher than that of dense models.
- **Balanced Trade-off (Elastic Net):** Elastic Net retains an average of 218 genes, achieving $99.81\% \pm 0.30\%$ accuracy and a lower log loss of 0.020. Compared to dense models, it uses fewer than a quarter of the features with essentially no loss in predictive performance.
- **Dense Models:** Dense linear models, tree ensembles, and standard neural networks achieve low log loss (0.004 to 0.016) but require evaluating all 1,000 features (or 434 principal components in PCA-MLP).

4.3 Feature Selection Stability

In applications where models are used to identify biological markers, it is important that the selected features are consistent across different samples. If a model selects completely different features from one split to another, the identified gene panel cannot be considered reliable.

We measured selection stability across the 10 data splits using the mean pairwise Jaccard index ($J = \frac{|S_i \cap S_j|}{|S_i \cup S_j|}$) and Dice coefficient (Table 3 and Figure 6). For tree models, we evaluated stability on the top 50 features ranked by importance score.

Table 3: Feature selection stability and signature size across 10 repeated resampling splits.

Model	Mean Signature Size (k)	Pairwise Jaccard Index	Pairwise Dice Index
ElasticNet	217.5 ± 6.5	$\mathbf{0.773 \pm 0.041}$	$\mathbf{0.872 \pm 0.026}$
Lasso	88.9 ± 3.2	$\mathbf{0.740 \pm 0.051}$	$\mathbf{0.849 \pm 0.033}$
XGBoost	50.0 ± 0.0	0.467 ± 0.044	0.635 ± 0.041
RandomForest	50.0 ± 0.0	0.242 ± 0.052	0.387 ± 0.069

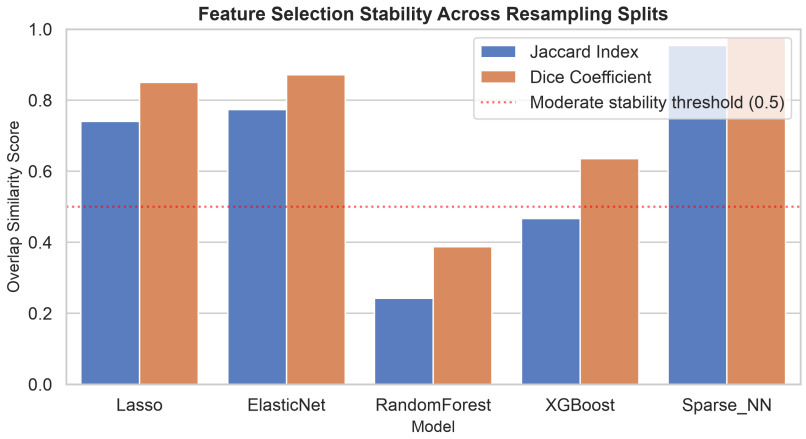


Figure 6: Feature selection stability (Jaccard and Dice indices) across resampling splits.

The stability results show a clear contrast between linear and tree models:

- **Linear Regularizers Are More Stable:** Elastic Net achieved the highest selection stability ($J = 0.773 \pm 0.041$), followed closely by Lasso ($J = 0.740 \pm 0.051$). Elastic Net's L_2 penalty helps keep correlated genes together, which provides slightly better stability than pure L_1 selection.
- **Tree Ensembles Show High Split Instability:** Despite achieving over 99.4% accuracy, the top 50 features selected by Random Forest had a mean pairwise Jaccard overlap of only **24.2%** ($J = 0.242 \pm 0.052$). XGBoost was somewhat better but still modest ($J = 0.467 \pm 0.044$).

  This instability happens because many genes in the dataset are strongly correlated ($r > 0.85$). At each decision node, multiple collinear genes provide nearly identical impurity reductions. The algorithm picks one gene essentially at random, while slightly different splits in the training fold lead to different choices. While this does not harm prediction accuracy, it means that feature importance rankings from tree models on this dataset are not stable across different data splits.

We also identified individual genes that were selected by Lasso and Elastic Net in every one of the 10 splits:

- **Consistently Selected by Lasso (10/10 folds):** `gene_9544`, `gene_8348`, `gene_12848`, `gene_7560`, `gene_12078`.
- **Consistently Selected by Elastic Net (10/10 folds):** `gene_12568`, `gene_14089`, `gene_8891`, `gene_13202`, `gene_18`.

Because the dataset uses anonymized gene identifiers, we cannot confirm the biological roles of these specific genes without additional mapping. Statistically, however, they represent features that were consistently retained by the regularizer across different patient partitions.

4.4 Sample Efficiency Stress Test

To see how models behave when training data is scarce, we evaluated performance across five training sample sizes:

$$n_{\text{train}} \in \{30, 60, 120, 250, 640\} \tag{3}$$

tested on the fixed outer test set of $N_{\text{test}} = 161$ samples. At $n_{\text{train}} = 30$, each model had only 6 training samples per cancer class across 1,000 dimensions.

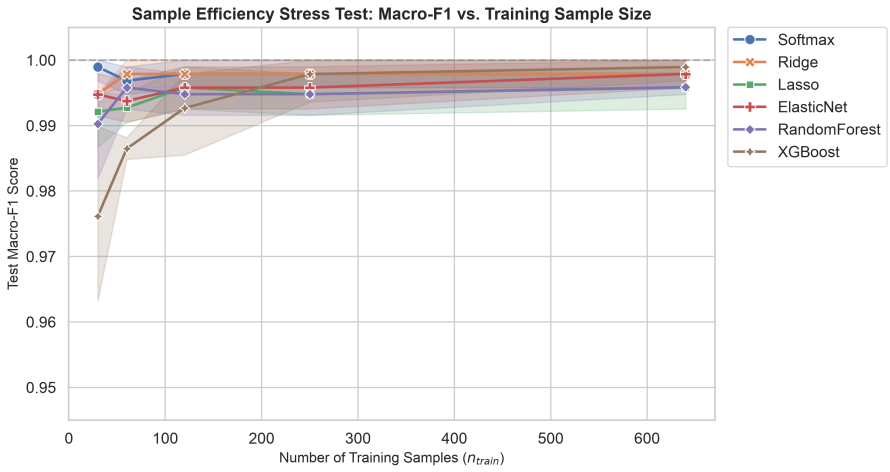


Figure 7: Sample efficiency comparison: Macro-F1 performance as a function of training sample size n_{train}.

Figure 7 shows how performance changes as sample size decreases:

- **Performance of Gradient Boosting with Few Samples:** XGBoost showed noticeable degradation at $n_{\text{train}} = 30$, dropping to 0.976 ± 0.018 **Macro-F1**, with a minimum fold score of 0.954. Random Forest also dropped slightly to 0.985. When sample sizes are small, greedy recursive tree splits are more likely to overfit noise in the training set.
- **Resilience of Regularized Linear Models:** In contrast, regularized linear models (Ridge, Softmax, and Elastic Net) maintained 0.995 ± 0.004 **Macro-F1** even at $n_{\text{train}} = 30$. The linear inductive bias combined with regularization prevented severe overfitting when data was limited.

5 Answers to Research Questions

Q1. Performance in the $p \gg n$ Regime: Moving from regularized linear models to non-linear tree ensembles or neural networks did not produce any meaningful improvement in classification accuracy or Macro-F1 on this dataset (0.998 vs. 0.998). Because the five tumor types are largely linearly separable in expression space, linear decision boundaries are sufficient for classification. Adding non-linear capacity increased model complexity without providing noticeable predictive gains.

Q2. Feature Selection Stability vs. Flexibility: Model flexibility came at the expense of feature selection stability. While tree ensembles achieved high accuracy, their feature importances varied considerably across splits ($J = 0.242$ for Random Forest, $J = 0.467$ for XGBoost) due to gene collinearity. In contrast, sparse linear models achieved much higher stability ($J = 0.773$ for Elastic Net, $J = 0.740$ for Lasso), with Elastic Net providing the most consistent feature sets.

Q3. Behavior under Sample Scarcity: When training data was reduced to $n = 30$ (6 samples per class), regularized linear models remained reliable ($F_1 \geq 0.995$), while gradient boosted trees degraded to $F_1 = 0.976$. Regularized linear models benefited from a strong inductive bias that avoided overfitting when training data was scarce.

6 Conclusion and Limitations

In this course project, we benchmarked eight classification models across three model families on the TCGA Pan-Cancer RNA-Seq dataset. Our main findings are:

1. Regularized linear models performed as well as or better than more complex non-linear models in accuracy, calibration, and training runtime.
2. Elastic Net logistic regression provided a good balance of feature sparsity (218 genes), probability calibration (log loss 0.020), and selection stability across data splits ($J = 0.773$).
3. Decision tree ensembles showed high feature selection instability under collinearity, making their feature importance rankings less dependable as consistent biomarker sets.
4. Linear regularizers handled small sample training ($n = 30$) more robustly than gradient boosting.

Limitations: Several limitations should be noted:

- We evaluated models on a single dataset of 801 samples across five distinct adult solid tumor types. We cannot conclude that these findings would carry over directly to more subtle classification tasks, such as distinguishing fine tumor subtypes or predicting clinical outcomes.
- The dataset features are anonymized gene labels, which prevented us from verifying whether the selected genes correspond to known cancer pathways.
- True clinical validation would require external test cohorts from different sequencing platforms and institutions, which was outside the scope of our project.

Within the scope of our benchmark, however, the results show that when data is linearly separable and features are correlated, simple regularized linear models provide practical advantages in interpretability, stability, and computational simplicity.

References

[1] S. Fiorini. gene expression cancer RNA-Seq. *UCI Machine Learning Repository*, 2016. https://doi.org/10.24432/C5R88H (CC BY 4.0; derived from TCGA PANCAN HiSeq data).

[2] J. N. Weinstein, E. A. Collisson, G. B. Mills, et al. The Cancer Genome Atlas Pan-Cancer analysis project. *Nature Genetics*, 45(10):1113–1120, 2013.

[3] H. Zou and T. Hastie. Regularization and variable selection via the elastic net. *Journal of the Royal Statistical Society: Series B*, 67(2):301–320, 2005.

[4] R. Tibshirani. Regression shrinkage and selection via the lasso. *Journal of the Royal Statistical Society: Series B*, 58(1):267–288, 1996.

[5] T. Chen and C. Guestrin. XGBoost: A scalable tree boosting system. In *ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794, 2016.